

Tugas 3 – Implementasi RESTful & GraphQL API

Studi Kasus: Aplikasi Wisata (WisataKu)

Nama: Sulistiani Nasya Marzu (Nasya) | NIM: SI21240018

Program Studi Sistem Informasi – STMIK Lombok

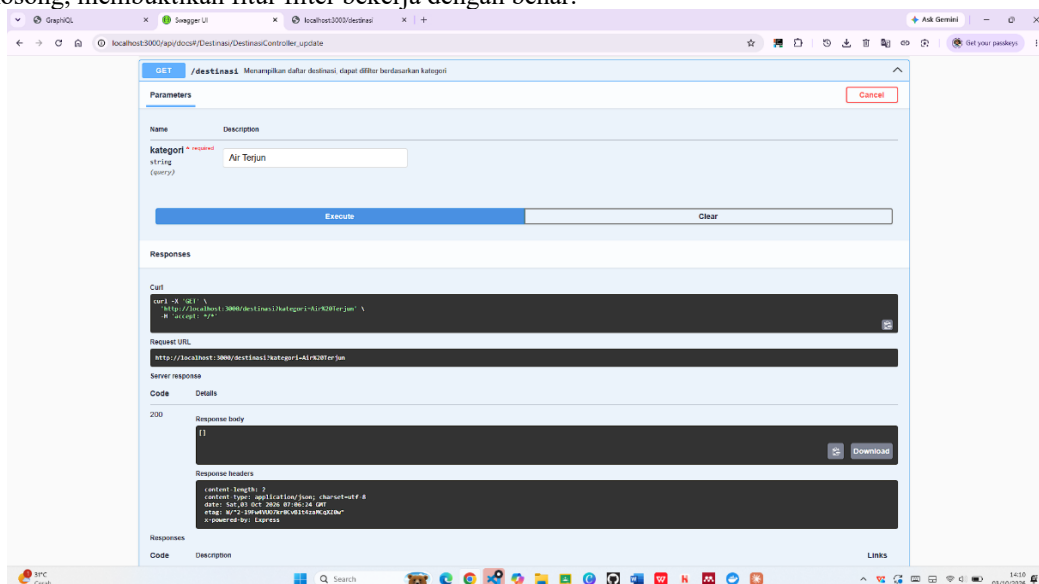
Ringkasan Pekerjaan

Tugas ini terdiri atas dua bagian, yaitu implementasi RESTful API dan GraphQL API untuk resource Destinasi pada aplikasi WisataKu. Bagian REST mencakup operasi CRUD (Create, Read, Update, Delete) yang terhubung langsung ke basis data MariaDB melalui Prisma ORM, sedangkan bagian GraphQL menyediakan Query dan Mutation yang memungkinkan pengambilan data gabungan (Destinasi, Ulasan, dan Fasilitas) dalam satu permintaan saja, sesuai konsep resolver dan nested field pada GraphQL.

Bagian A — Implementasi RESTful API

1. GET /destinasi (dengan filter kategori)

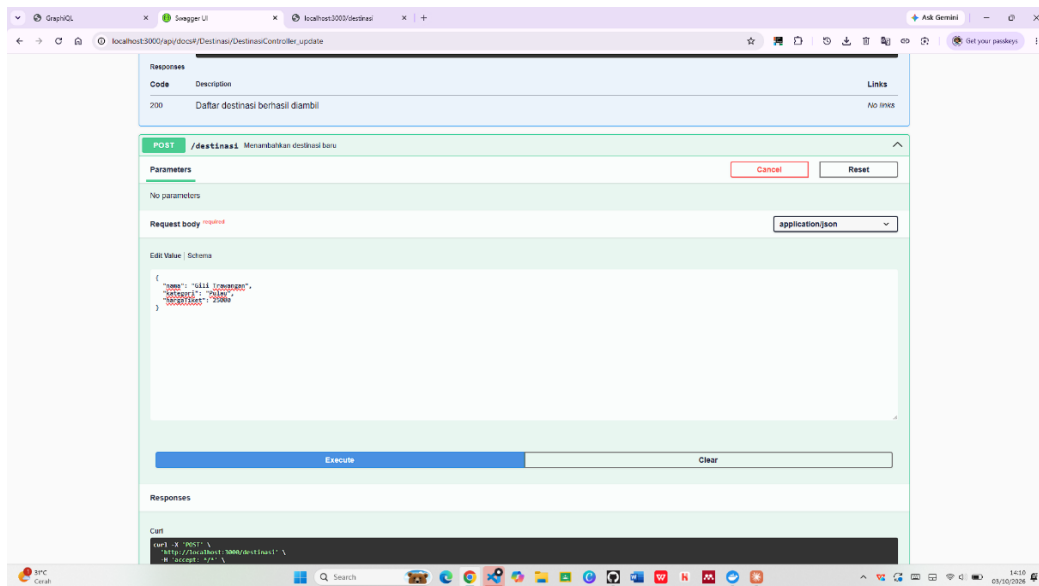
Pengujian endpoint GET /destinasi dengan parameter query kategori untuk memfilter data destinasi berdasarkan kategorinya. Pada contoh ini, kategori "Air Terjun" belum memiliki data sehingga response mengembalikan array kosong, membuktikan fitur filter bekerja dengan benar.



Gambar 1. GET /destinasi dengan filter kategori

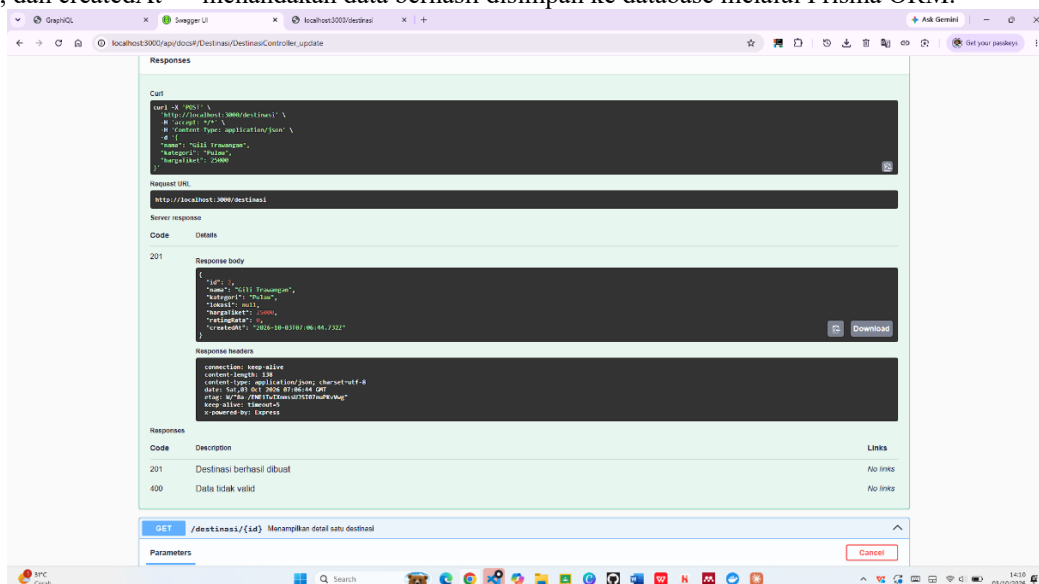
2. POST /destinasi

Pengujian menambahkan data destinasi baru ("Gili Trawangan", kategori Pulau, harga tiket Rp25.000) melalui request body sesuai skema CreateDestinasiDto.



Gambar 2. Request body POST /destinasi

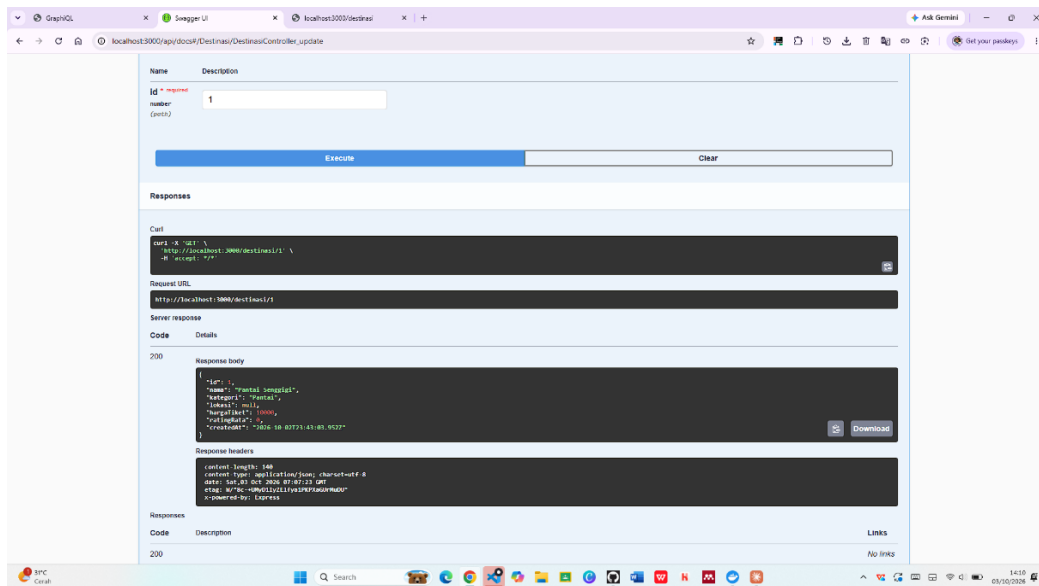
Server merespons dengan status 201 (Created) dan mengembalikan data destinasi baru beserta id, ratingRata default, dan createdAt — menandakan data berhasil disimpan ke database melalui Prisma ORM.



Gambar 3. Response POST /destinasi – status 201 (berhasil dibuat)

3. GET /destinasi/{id}

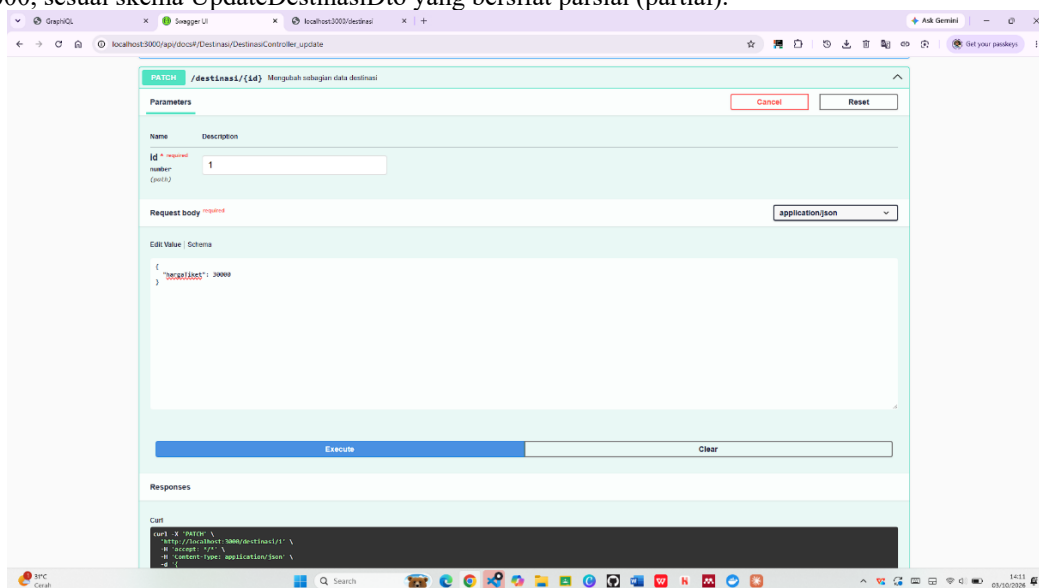
Pengujian mengambil detail satu data destinasi (id: 1) untuk memverifikasi bahwa data tersimpan dengan struktur yang sesuai. Server merespons dengan status 200 (OK).



Gambar 4. GET /destinasi/1 – status 200

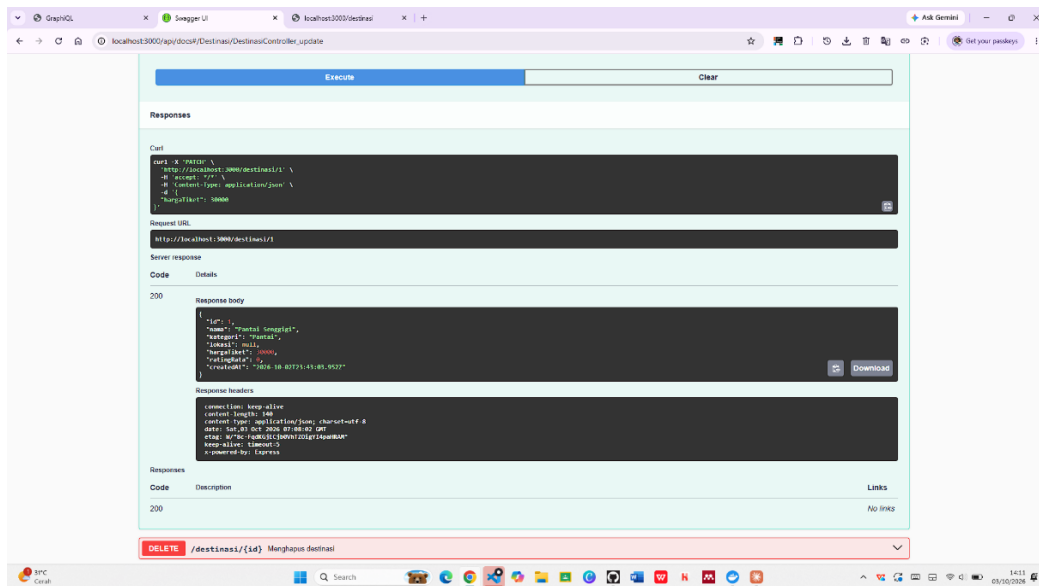
4. PATCH /destinasi/{id}

Pengujian mengubah sebagian data destinasi (id: 1), yaitu mengubah hargaTiket dari Rp10.000 menjadi Rp30.000, sesuai skema UpdateDestinasiDto yang bersifat parsial (partial).



Gambar 5. Request body PATCH /destinasi/1

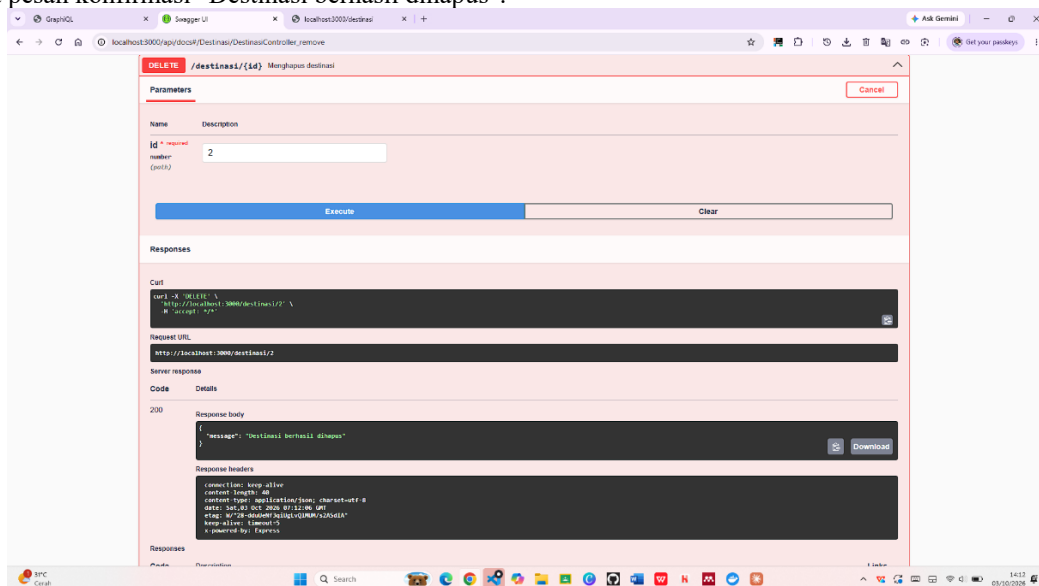
Server merespons dengan status 200 dan mengembalikan data destinasi yang telah diperbarui, membuktikan hargaTiket berhasil berubah menjadi 30000.



Gambar 6. Response PATCH /destinasi/1 – data berhasil diperbarui

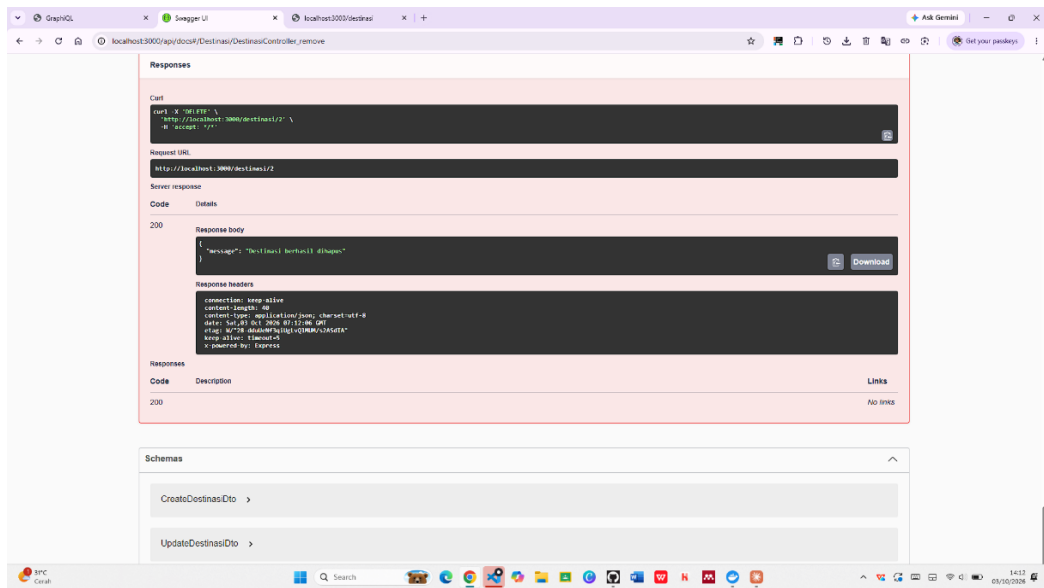
5. DELETE /destinasi/{id}

Pengujian menghapus data destinasi dengan id: 2 ("Gili Trawangan"). Server merespons dengan status 200 beserta pesan konfirmasi "Destinasi berhasil dihapus".



Gambar 7. Response DELETE /destinasi/2 – berhasil dihapus

Untuk memastikan penghapusan berhasil, dilakukan pengecekan ulang pada endpoint GET /destinasi, dan terbukti data dengan id 2 sudah tidak muncul lagi, hanya tersisa data dengan id 1.



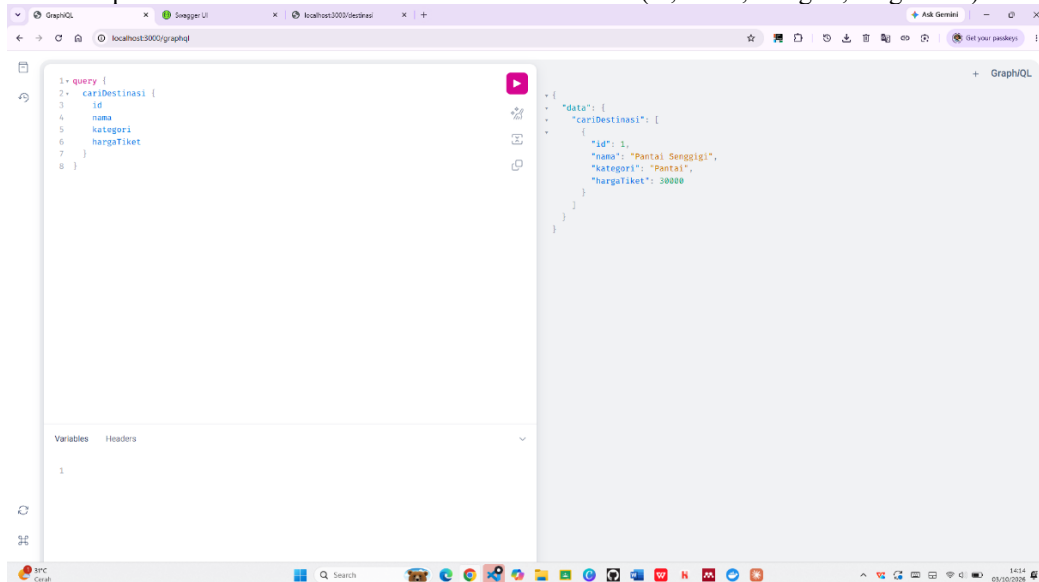
Gambar 8. GET /destinasi setelah penghapusan – hanya tersisa id 1

Bagian B — Implementasi GraphQL API

GraphQL diimplementasikan menggunakan pendekatan code-first melalui `@nestjs/graphql` dan Apollo Server, dapat diakses melalui Apollo Sandbox / GraphiQL pada endpoint `/graphql`. Keunggulan utama GraphQL dibandingkan REST adalah kemampuannya mengambil data relasi (Ulasan dan Fasilitas) dalam satu kali permintaan, tanpa perlu melakukan beberapa request terpisah seperti pada REST API.

1. Query `cariDestinasi`

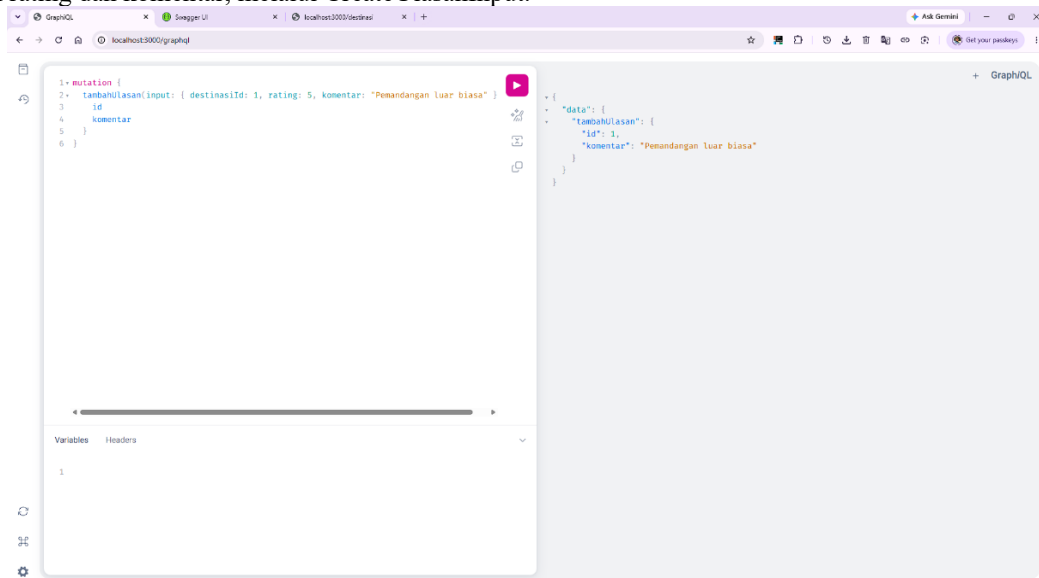
Query ini menampilkan daftar seluruh destinasi beserta field dasar (`id`, `nama`, `kategori`, `hargaTiket`).



Gambar 9. Query `cariDestinasi`

2. Mutation `tambahUlasan`

Mutation ini digunakan untuk menambahkan data ulasan baru pada sebuah destinasi (`destinasiId: 1`), lengkap dengan rating dan komentar, melalui `CreateUlasanInput`.

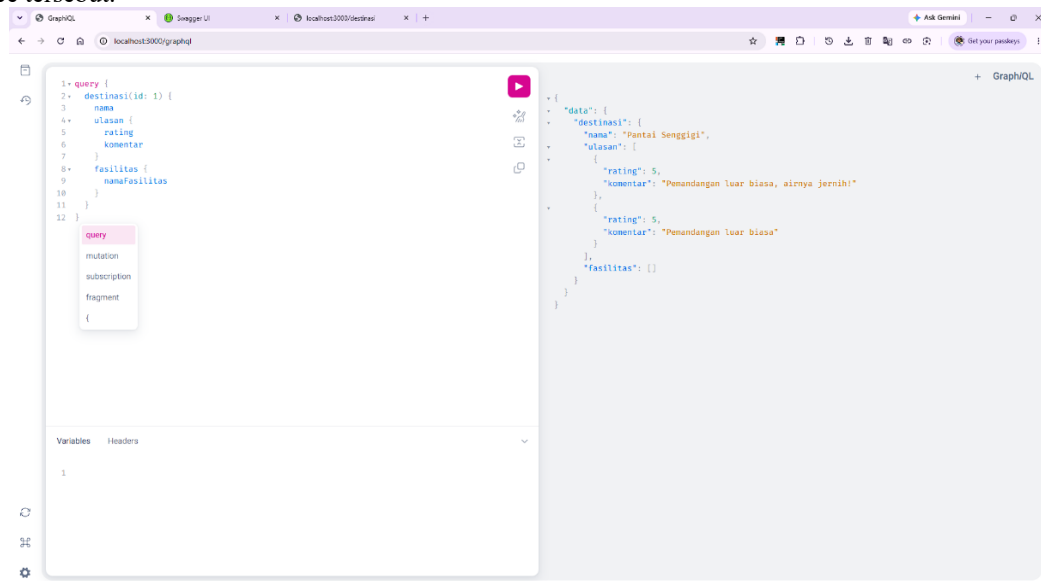


Gambar 10. Mutation `tambahUlasan` – berhasil menambahkan ulasan baru

3. Query Data Gabungan (Destinasi + Ulasan + Fasilitas)

Query berikut membuktikan keunggulan GraphQL: dalam satu permintaan, client dapat mengambil data destinasi sekaligus daftar ulasan (hasil dari mutation sebelumnya) dan fasilitasnya melalui nested field, memanfaatkan `@ResolveField` pada `DestinasiResolver`. Terlihat field ulasan kini berisi data ulasan yang telah

ditambahkan, sedangkan fasilitas tetap berupa array kosong karena belum ada data yang ditambahkan untuk resource tersebut.



Gambar 11. Query destinasi(id: 1) dengan nested field ulasan dan fasilitas

Perbandingan Jumlah Request REST vs GraphQL

Untuk menampilkan halaman detail destinasi secara lengkap (nama, kategori, daftar ulasan, dan daftar fasilitas), pendekatan REST murni membutuhkan tiga kali request terpisah, yaitu ke endpoint GET /destinasi/{id}, GET /destinasi/{id}/ulasan, dan GET /destinasi/{id}/fasilitas. Sebaliknya, dengan GraphQL, seluruh data tersebut dapat diperoleh hanya dengan satu kali request melalui query destinasi dengan nested field ulasan dan fasilitas, sehingga lebih efisien dari sisi jumlah round-trip ke server.